ICT4153

Mobile Application Development

Practical 09

Flutter Form Validation & Gestures

Objective

To cover form validation, form submission, and gestures in Flutter using the resources provided.

1. Creating and Styling Text Fields

Text fields are fundamental widgets that allow users to input text. Flutter provides the TextField and TextFormField widgets for this purpose.

- **TextField**: A basic text input field

Example:

```
TextField(
  decoration: InputDecoration(
    border: OutlineInputBorder(),
    hintText: 'Enter your name',
  ),
)
```

TextFormField: A variant of TextField that integrates with the Form widget, providing additional functionalities like input validation.

Example:

```
TextFormField(
  decoration: InputDecoration(
    labelText: 'Enter your email',
    border: UnderlineInputBorder(),
  ),
  validator: (value) {
    if (value == null || value.isEmpty) {
      return 'Please enter your email';
    }
```

```
    return null;
  },
)
```

2. Retrieving and Handling User Input

To retrieve the text entered by the user, utilize a TextEditingController.

- Steps:

1. Create a TextEditingController instance.
2. Assign it to the controller property of the TextField or TextFormField.
3. Access the text via controller.text.

Example:

```
final TextEditingController _controller = TextEditingController();

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}

TextField(
  controller: _controller,
  decoration: InputDecoration(
    labelText: 'Enter your message',
  ),
),

ElevatedButton(
  onPressed: () {
    print('User input: ${_controller.text}');
  },
  child: Text('Submit'),
),
```

Building a Form with Validation

Forms are essential for grouping multiple input fields and validating user input.

- Steps:

1. Wrap your input fields with a Form widget.
2. Assign a GlobalKey<FormState> to the form to manage its state.
3. Use TextFormField widgets with validator functions to define validation logic.
4. Call _formKey.currentState!.validate() to trigger validation.

Example:

```
final GlobalKey<FormState> _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: Column(
    children: [
      TextFormField(
        decoration: InputDecoration(labelText: 'Username'),
        validator: (value) {
          if (value == null || value.isEmpty) {
            return 'Please enter your username';
          }
          return null;
        },
      ),
      TextFormField(
        decoration: InputDecoration(labelText: 'Password'),
        obscureText: true,
        validator: (value) {
          if (value == null || value.isEmpty) {
            return 'Please enter your password';
          }
          return null;
        },
      ),
      ElevatedButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) {
            // Process data
          }
        },
        child: Text('Submit'),
      ),
    ],
  ),
);
```

3. Designing a Form Submission Page

A form submission page typically includes multiple input fields and a submit button.

Example:

```
import 'package:flutter/material.dart';

class MyFormPage extends StatefulWidget {
  @override
  MyFormPageState createState() => MyFormPageState();
}

class MyFormPageState extends State<MyFormPage> {
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
  final TextEditingController _nameController = TextEditingController();
  final TextEditingController _emailController = TextEditingController();

  @override
  void dispose() {
    _nameController.dispose();
    _emailController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Form Submission')),
      body: Padding(
        padding: EdgeInsets.all(16.0),
        child: Form(
          key: _formKey,
          child: Column(
            children: [
              TextFormField(
                controller: _nameController,
                decoration: InputDecoration(labelText: 'Name'),
                validator: (value) {
                  if (value == null || value.isEmpty) {
                    return 'Please enter your name';
                  }
                  return null;
                },
              ),
              TextFormField(
                controller: _emailController,
                decoration: InputDecoration(labelText: 'Email'),
                validator: (value) {
```

```
              if (value == null || value.isEmpty) {
                return 'Please enter your email';
              }
              return null;
            },
          ),
          ElevatedButton(
            onPressed: () {
              if (_formKey.currentState!.validate()) {
                // Process data
              }
            },
            child: Text('Submit'),
          ),
        ],
      ),
    ),
  ),
 );
 }
}
```

4. Implementing Gesture Detection

Gestures like taps, swipes, and pinches enhance user interaction. Flutter's GestureDetector widget detects various gestures.

Example:

```
GestureDetector(
 onTap: () {
  print('Container tapped');
 },
 child: Container(
  color: Colors.blue,
  width: 100,
  height: 100,
  child: Center(child: Text('Tap me')),
 ),
)
```

5. Complete Example: Form with Gesture Interaction

Combining form validation and gesture detection provides a comprehensive user input experience.

Example:

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget
```
::contentReference[oaicite:42]{index=42}

Activity 01

Task Requirements

1. Form Validation
    - Create a form with fields: Name, Email, Password, Confirm Password.
    - Validate each field:
        - Name: Must not be empty.
        - Email: Must be a valid email (contains @).
        - Password: Must be at least 6 characters.
        - Confirm Password: Must match the Password field.
    - Show error messages if validation fails.
    - On successful submission, display a SnackBar with a success message.

2. Gesture Interactions
    - Add a "Clear Form" button that resets all fields when long-pressed.
    - Make the Submit button show a ripple effect (InkWell).

3. Additon
    - Add a profile picture upload option using GestureDetector (tap to select an image).
    - Show a confirmation dialog before form submission.

Implementation Steps

1. Create a New Flutter Project

```
flutter create form_validation_task cd
form_validation_task
```

Replace lib/main.dart with the Following Code

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: UserRegistrationForm(),
    );
  }
}

class UserRegistrationForm extends
StatefulWidget {
  @override
  UserRegistrationFormState createState() =>
UserRegistrationFormState();
}

class UserRegistrationFormState extends
State<UserRegistrationForm> {
  final GlobalKey<FormState> _formKey =
GlobalKey<FormState>();
  final TextEditingController _nameController =
TextEditingController();
  final TextEditingController _emailController =
TextEditingController();
  final TextEditingController
_passwordController = TextEditingController();
  final TextEditingController
_confirmPasswordController =
TextEditingController();

  void _submitForm() {
    if (_formKey.currentState!.validate()) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Registration
```

```
Successful!')),
    );
    print('Name: ${_nameController.text}');
    print('Email: ${_emailController.text}');
  }
 }

 void _clearForm() {
  _nameController.clear();
  _emailController.clear();
  _passwordController.clear();
  _confirmPasswordController.clear();
 }

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(title: Text('User
Registration')),
   body: Padding(
    padding: EdgeInsets.all(16.0),
    child: Form(
     key: _formKey,
     child: Column(
      children: [
       TextFormField(
        controller: _nameController,
        decoration: InputDecoration(labelText:
'Name'),
        validator: (value) {
         if (value == null || value.isEmpty) {
          return 'Please enter your name';
         }
         return null;
        },
       ),
       TextFormField(
        controller: _emailController,
        decoration: InputDecoration(labelText:
'Email'),
        validator: (value) {
         if (value == null || value.isEmpty) {
          return 'Please enter your email';
         }
         if (!value.contains('@')) {
          return 'Enter a valid email';
         }
```

```
            return null;
          },
        ),
        TextFormField(
          controller: _passwordController,
          decoration: InputDecoration(labelText:
'Password'),
          obscureText: true,
          validator: (value) {
            if (value == null || value.isEmpty) {
              return 'Please enter a password';
            }
            if (value.length < 6) {
              return 'Password must be at least 6
characters';
            }
            return null;
          },
        ),
        TextFormField(
          controller:
_confirmPasswordController,
          decoration: InputDecoration(labelText:
'Confirm Password'),
          obscureText: true,
          validator: (value) {
            if (value == null || value.isEmpty) {
              return 'Please confirm password';
            }
            if (value != _passwordController.text) {
              return 'Passwords do not match';
            }
            return null;
          },
        ),
        SizedBox(height: 20),
        Row(
          mainAxisAlignment:
MainAxisAlignment.spaceEvenly,
          children: [
            GestureDetector(
              onLongPress: _clearForm,
              child: Container(
                padding: EdgeInsets.all(12),
                color: Colors.grey,
                child: Text('Long Press to Clear'),
              ),
```

```
              ),
              InkWell(
                onTap: _submitForm,
                borderRadius:
BorderRadius.circular(8),
                child: Container(
                  padding: EdgeInsets.all(12),
                  color: Colors.blue,
                  child: Text('Submit', style:
TextStyle(color: Colors.white)),
                ),
              ),
            ],
          ),
        ],
      ),
    ),
  );
 }
}
```

Expected Output

1. Form Validation
If fields are empty/invalid:
Validation Error

On successful submission:
Success SnackBar

2. Gesture Interactions
Long Press on "Clear" button resets the form.

Submit button shows a ripple effect.

Task Submission

- Run the app and test all validations.
- Modify the UI (add a profile picture upload if attempting the additional section).
- Submit your code (GitHub link or .zip file) to the given link in LMS.

Flutter – Navigation and Routing

1. Basic Navigation Between Screens

Key Concepts:

- Navigator.push() - Add a new route to the stack
- Navigator.pop() - Remove the current route from the stack
- MaterialPageRoute - Standard page transition

```
import 'package:flutter/material.dart';

void main() => runApp(NavigationApp());

class NavigationApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: FirstScreen(),
    );
  }
}

class FirstScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('First Screen')),
      body: Center(
        child: ElevatedButton(
          child: Text('Go to Second Screen'),
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => SecondScreen()),
            );
          },
        ),
      ),
    );
  }
}

class SecondScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(title: Text('Second Screen')),
      body: Center(
        child: ElevatedButton(
          child: Text('Go Back'),
          onPressed: () {
            Navigator.pop(context);
          },
        ),
      ),
    );
  }
}
```

2. Passing Data Between Screens

Key Concepts:

- Passing data when navigating to a new screen
- Receiving data in the new screen

```
import 'package:flutter/material.dart';

void main() => runApp(DataPassingApp());

class DataPassingApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: HomeScreen());
  }
}

class HomeScreen extends StatelessWidget {
  final List<String> items = ['Apple', 'Banana', 'Orange'];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Home Screen')),
      body: ListView.builder(
        itemCount: items.length,
        itemBuilder: (context, index) {
          return ListTile(
            title: Text(items[index]),
            onTap: () {
```

```
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) => DetailScreen(item: items[index]),
              ),
            );
          },
        );
      },
    ),
  );
 }
}

class DetailScreen extends StatelessWidget {
 final String item;
 DetailScreen({required this.item});

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(title: Text('Detail Screen')),
   body: Center(child: Text('Selected Item: $item')),
  );
 }
}
```

3. Named Routes

Key Concepts:

- Defining routes in MaterialApp
- Navigating using route names
- Passing arguments with named routes

```
import 'package:flutter/material.dart';

void main() => runApp(NamedRoutesApp());

class NamedRoutesApp extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
```

```
    return MaterialApp(
      initialRoute: '/',
      routes: {
        '/': (context) => MainScreen(),
        '/settings': (context) => SettingsScreen(),
        '/profile': (context) => ProfileScreen(name: 'John Doe'),
      },
    );
  }
}

class MainScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Main Screen')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton(
              child: Text('Go to Settings'),
              onPressed: () {
                Navigator.pushNamed(context, '/settings');
              },
            ),
            ElevatedButton(
              child: Text('View Profile'),
              onPressed: () {
                Navigator.pushNamed(context, '/profile');
              },
            ),
          ],
        ),
      ),
    );
  }
}

class SettingsScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Settings')),
      body: Center(
        child: ElevatedButton(
          child: Text('Go Back'),
```

```
        onPressed: () {
          Navigator.pop(context);
        },
      ),
    ),
  );
 }
}

class ProfileScreen extends StatelessWidget {
 final String name;
 ProfileScreen({required this.name});

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(title: Text('Profile')),
   body: Center(child: Text('Welcome, $name')),
  );
 }
}
```

Tabs in Flutter

1. Introduction to Tabs in Flutter

In Flutter, tabs are implemented using the TabBar and TabBarView widgets in conjunction with a TabController. The TabBar displays the tabs, while the TabBarView displays the corresponding content for each tab. The TabController synchronizes the user's interaction with the tabs and the content displayed.

2. Setting Up the Project

Begin by creating a new Flutter project. You can do this by running the following command in your terminal:

```
flutter create tabbed_app
```

Navigate into the project directory:

```
cd tabbed_app
```

3. Implementing the Tabbed Interface

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Tabs Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const MyHomePage(),
    );
  }
}

class MyHomePage extends StatelessWidget {
  const MyHomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return DefaultTabController(
      length: 3,
      child: Scaffold(
        appBar: AppBar(
          title: const Text('Flutter Tabs Demo'),
          bottom: const TabBar(
            tabs: [
              Tab(icon: Icon(Icons.directions_car), text: 'Car'),
              Tab(icon: Icon(Icons.directions_transit), text: 'Transit'),
              Tab(icon: Icon(Icons.directions_bike), text: 'Bike'),
            ],
          ),
        ),
        body: const TabBarView(
          children: [
            Center(child: Text('Car Tab Content')),
            Center(child: Text('Transit Tab Content')),
```

```
            Center(child: Text('Bike Tab Content')),
          ],
        ),
      ),
    );
  }
}
```

4. Understanding the Code

- DefaultTabController: This widget provides a default TabController for the TabBar and TabBarView. The length parameter specifies the number of tabs.Flutter documentation
- AppBar: Contains the TabBar widget, which defines the tabs. Each Tab can have an icon and/or text.Flutter documentation
- TabBarView: Displays the content corresponding to each tab. The order of the widgets in the children list matches the order of the tabs in the TabBar.

5. Running the Application

To run the application, use the following command:

```
flutter run
```

This will launch the application on your connected device or emulator. You should see an app bar with three tabs labeled "Car," "Transit," and "Bike." Tapping on each tab will display the corresponding content below.

6. Customizing the Tabs

You can customize the appearance and behavior of the tabs by modifying the TabBar and TabBarView widgets. For example, to change the color of the selected tab, you can set the indicatorColor property of the TabBar:

```
bottom: const TabBar(
  indicatorColor: Colors.red,
  tabs: [
    Tab(icon: Icon(Icons.directions_car), text: 'Car'),
    Tab(icon: Icon(Icons.directions_transit), text: 'Transit'),
    Tab(icon: Icon(Icons.directions_bike), text: 'Bike'),
  ],
```

```
),
```

For more advanced customization options, refer to the Flutter documentation on working with tabs.

Navigate to a new screen and back

1. Basic Navigation Between Two Screens

Key Concepts

- Navigator.push() - Adds a route to the stack
- Navigator.pop() - Removes the current route
- MaterialPageRoute - Standard material design page transition

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Navigation Basics',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const FirstScreen(),
    );
  }
}

class FirstScreen extends StatelessWidget {
  const FirstScreen({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('First Screen'),
      ),
      body: Center(
        child: ElevatedButton(
          child: const Text('Launch Second Screen'),
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => const SecondScreen()),
            );
          },
        ),
      ),
    );
  }
}

class SecondScreen extends StatelessWidget {
  const SecondScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Second Screen'),
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.pop(context);
          },
          child: const Text('Go back!'),
        ),
      ),
    );
  }
```

```
}
```

How It Works

1. The app starts with FirstScreen
2. Tapping the button uses Navigator.push() to show SecondScreen
3. The back button in SecondScreen uses Navigator.pop() to return.

2. Passing Data to a New Screen

Key Concepts

- Pass data via constructor when creating a new route
- Receive and display the data in the new screen

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Passing Data',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: TodosScreen(
        todos: List.generate(
          3,
          (i) => Todo(
            title: 'Todo $i',
            description: 'Description of what needs to be done for Todo $i',
          ),
        ),
      ),
    );
  }
}
```

```
class Todo {
  final String title;
  final String description;

  const Todo({required this.title, required this.description});
}

class TodosScreen extends StatelessWidget {
  final List<Todo> todos;
  const TodosScreen({super.key, required this.todos});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Todos')),
      body: ListView.builder(
        itemCount: todos.length,
        itemBuilder: (context, index) {
          return ListTile(
            title: Text(todos[index].title),
            onTap: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => DetailScreen(todo: todos[index]),
                ),
              );
            },
          );
        },
      ),
    );
  }
}

class DetailScreen extends StatelessWidget {
  final Todo todo;
  const DetailScreen({super.key, required this.todo});

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(title: Text(todo.title)),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Text(todo.description),
      ),
    );
  }
}
```

How It Works

- TodosScreen displays a list of todos
- Tapping a todo navigates to DetailScreen with the selected todo data
- DetailScreen displays the todo's title and description

3. Returning Data from a Screen

Key Concepts

- Use Navigator.push() with await to wait for a result
- Return data using Navigator.pop()

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(
    title: 'Returning Data',
    home: HomeScreen(),
  ));
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Returning Data Demo')),
      body: const Center(child: SelectionButton()),
    );
  }
}
class SelectionButton extends StatefulWidget {
```

```
  const SelectionButton({super.key});

  @override
  State<SelectionButton> createState() => _SelectionButtonState();
}

class _SelectionButtonState extends State<SelectionButton> {
  String? _selectedOption;

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        ElevatedButton(
          onPressed: () {
            _navigateAndDisplaySelection(context);
          },
          child: const Text('Pick an option'),
        ),
        const SizedBox(height: 20),
        Text(
          _selectedOption ?? 'No option selected',
          style: const TextStyle(fontSize: 20),
        ),
      ],
    );
  }

  Future<void> _navigateAndDisplaySelection(BuildContext context) async {
    final result = await Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => const SelectionScreen()),
    );

    if (!mounted) return;

    setState(() {
      _selectedOption = result;
    });
  }
}

class SelectionScreen extends StatelessWidget {
  const SelectionScreen({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Pick an option')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton(
              onPressed: () {
                Navigator.pop(context, 'Yes');
              },
              child: const Text('Yes'),
            ),
            ElevatedButton(
              onPressed: () {
                Navigator.pop(context, 'No');
              },
              child: const Text('No'),
            ),
          ],
        ),
      ),
    );
  }
}
```

How It Works

1. HomeScreen has a button to navigate to SelectionScreen
2. SelectionScreen has two buttons that return different values
3. The selected value is displayed back in HomeScreen

Send data to a new screen and Return data from a screen

Effective navigation and data management between screens are fundamental aspects of building robust Flutter applications. This practical guide will walk you through the processes of passing data to a new screen and returning data from a screen in Flutter, providing comprehensive explanations and complete code examples.

1. Passing Data to a New Screen

When developing Flutter applications, it's common to navigate to a new screen and pass data to it. For instance, in a task management app, you might want to display details of a selected task on a new screen.

Steps to Pass Data to a New Screen:

- Define a Data Model: Create a class that represents the data you intend to pass.

```
class Task {
  final String title;
  final String description;

  Task({required this.title, required this.description});
}
```

Create the Destination Screen: Design the screen that will receive and display the data.

```
class TaskDetailScreen extends StatelessWidget {
  final Task task;

  const TaskDetailScreen({Key? key, required this.task}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(task.title),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Text(task.description),
      ),
    );
  }
}
```

- Navigate and Pass Data: Use the Navigator.push method to navigate to the new screen, passing the data as arguments.

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => TaskDetailScreen(task: selectedTask),
  ),
);
```

Complete Example:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const TaskApp());
}

class TaskApp extends StatelessWidget {
  const TaskApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Task Manager',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const TaskListScreen(),
    );
  }
}

// Data Model
class Task {
  final String title;
  final String description;

  const Task({required this.title, required this.description});
}

// Screen showing list of tasks
class TaskListScreen extends StatelessWidget {
  const TaskListScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final tasks = List.generate(
      10,
      (i) => Task(
        title: 'Task $i',
        description: 'Details of Task $i',
      ),
    );
```

```
    return Scaffold(
      appBar: AppBar(title: const Text('Tasks')),
      body: ListView.builder(
        itemCount: tasks.length,
        itemBuilder: (context, index) {
          return ListTile(
            title: Text(tasks[index].title),
            onTap: () async {
              // Navigate and wait for result
              final result = await Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => TaskDetailScreen(task: tasks[index]),
                ),
              );

              if (result != null) {
                ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(content: Text('Returned: $result')),
                );
              }
            },
          );
        },
      ),
    );
  }
}

// Destination screen showing task details
class TaskDetailScreen extends StatelessWidget {
  final Task task;

  const TaskDetailScreen({super.key, required this.task});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(task.title)),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
```

```
      child: Column(
       mainAxisAlignment: MainAxisAlignment.center,
       children: [
        Text(task.description),
        const SizedBox(height: 20),
        ElevatedButton(
         onPressed: () {
          // Return data back to previous screen
          Navigator.pop(context, 'Completed ${task.title}');
         },
         child: const Text('Mark as Completed'),
        ),
       ],
      ),
     ),
    );
  }
}
```

2. Returning Data from a Screen

In some scenarios, you may need to navigate to a new screen and receive data back upon its completion. For example, navigating to a selection screen and obtaining the user's choice.

Steps to Return Data from a Screen:

- Navigate to the New Screen and Await Result: Use the Navigator.push method and await the result using the async and await keywords.

```
final result = await Navigator.push(
 context,
 MaterialPageRoute(
  builder: (context) => SelectionScreen(),
 ),
);
```

- Return Data from the Second Screen: Use the Navigator.pop method to return data when the user completes an action.

```
Navigator.pop(context, selectedOption);
```

Complete Example:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const SelectionApp());
}

class SelectionApp extends StatelessWidget {
  const SelectionApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Selection Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const HomeScreen(),
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Home')),
      body: Center(
        child: ElevatedButton(
          onPressed: () async {
            // Navigate and wait for result
            final result = await Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) => const SelectionScreen(),
              ),
            );

            if (result != null) {
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(content: Text('You selected: $result')),
              );
```

```
          }
        },
        child: const Text('Pick an option'),
      ),
    ),
  );
 }
}

class SelectionScreen extends StatelessWidget {
 const SelectionScreen({super.key});

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(title: const Text('Select an Option')),
   body: Center(
    child: Column(
     mainAxisAlignment: MainAxisAlignment.center,
     children: <Widget>[
      ElevatedButton(
       onPressed: () {
        Navigator.pop(context, 'Option 1');
       },
       child: const Text('Option 1'),
      ),
      ElevatedButton(
       onPressed: () {
        Navigator.pop(context, 'Option 2');
       },
       child: const Text('Option 2'),
      ),
     ],
    ),
   ),
  );
 }
}
```

For more navigation and routing methods, please refer to the https://docs.flutter.dev/.